

```
import React, { useEffect, useState, useRef } from 'react';
import { NavigationContainer, DefaultTheme, DarkTheme, createNavigationContainerRef } from
 '@react-navigation/native';
import { createStackNavigator } from '@react-navigation/stack';
import { QueryClient, QueryClientProvider } from './src/lib/react-query';
import Toast, { BaseToast, ErrorToast, InfoToast, ToastConfig, BaseToastProps } from
 'react-native-toast-message';
import { StatusBar } from 'expo-status-bar';
import { registerRootComponent } from 'expo';
import { useColorScheme, View, Text, TouchableOpacity, Linking, AppState, Alert, ScrollView }
 from 'react-native';
import { useAuthStore } from './src/store/auth';
import { api, initApiUrl } from './src/lib/api';
import { useThemeStore, lightColors, darkColors } from './src/store/theme';
import { I18nProvider, useI18n } from './src/lib/i18n';
import * as SecureStore from 'expo-secure-store';
import * as Clipboard from 'expo-clipboard';
import AsyncStorage from '@react-native-async-storage/async-storage';
import { initAnalytics, logEvent, logScreenView, setUserId, setUserProperties } from
 './src/lib/analytics';
import { initNotifications, getPushToken } from './src/lib/notifications';
import { CollectionViewsProvider } from './src/lib/collectionViewsContext';

import LoginScreen from './src/screens/LoginScreen';
import MainTabNavigator from './src/navigation/MainTabNavigator';
import TermsScreen from './src/screens/TermsScreen';
import CollectionFormScreen from './src/screens/CollectionFormScreen';
import CollectionDetailScreen from './src/screens/CollectionDetailScreen';
import PlatformSelectScreen from './src/screens/PlatformSelectScreen';
import ShareManagementScreen from './src/screens/ShareManagementScreen';
import CreateShareScreen from './src/screens/CreateShareScreen';
import ShareDetailScreen from './src/screens/ShareDetailScreen';
import AccountSettingsScreen from './src/screens/AccountSettingsScreen';
import TagManageScreen from './src/screens/TagManageScreen';
import ManagementScreen from './src/screens/ManagementScreen';
import PlatformStatsScreen from './src/screens/PlatformStatsScreen';
import TierScreen from './src/screens/TierScreen';
import TierUpgradeScreen from './src/screens/TierUpgradeScreen';
import QuickSaveSettingsScreen from './src/screens/QuickSaveSettingsScreen';
import CollectionViewConfigScreen from './src/screens/CollectionViewConfigScreen';
import TrashScreen from './src/screens/TrashScreen';
import DuplicateDetectScreen from './src/screens/DuplicateDetectScreen';
import AutoBackupScreen from './src/screens/AutoBackupScreen';
import ExportScreen from './src/screens/ExportScreen';
```

```
const Stack = createStackNavigator();
const queryClient = new QueryClient();
const navigationRef = createNavigationContainerRef();

function getActiveRouteName(state: any): string | null {
  const route = state.routes[state.index];
  if (route.state) {
    return getActiveRouteName(route.state);
  }
  return route.name;
}

const toastConfig: ToastConfig = {
  success: (props: BaseToastProps) => (
    <BaseToast
      {...props}
      text1Style={{ fontSize: 16, fontWeight: '600' }}
      text2Style={{ fontSize: 14 }}
      style={{ borderLeftWidth: 4, borderLeftColor: '#5B8A72' }}
    />
  ),
  error: (props: BaseToastProps) => (
    <ErrorToast
      {...props}
      text1Style={{ fontSize: 16, fontWeight: '600' }}
      text2Style={{ fontSize: 14 }}
      style={{ borderLeftWidth: 4, borderLeftColor: '#B85C5C' }}
    />
  ),
  info: (props: BaseToastProps) => (
    <InfoToast
      {...props}
      text1Style={{ fontSize: 16, fontWeight: '600' }}
      text2Style={{ fontSize: 14 }}
      style={{ borderLeftWidth: 4, borderLeftColor: '#576F9F' }}
    />
  ),
};

class ErrorBoundaryInner extends React.Component<
  { children: React.ReactNode },
  { hasError: boolean; error: Error | null }
> {
```

```
state = { hasError: false, error: null as Error | null };

static getDerivedStateFromError(error: Error) {
  return { hasError: true, error };
}

handleReset = () => {
  this.setState({ hasError: false, error: null });
};

render() {
  if (this.state.hasError) {
    return <ErrorFallback error={this.state.error} onReset={this.handleReset} />;
  }
  return this.props.children;
}
}

function ErrorFallback({ error, onReset }: { error: Error | null; onReset: () => void }) {
  return (
    <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center', padding: 24,
backgroundColor: '#f5f5f5' }}>
      <Text style={{ fontSize: 48, marginBottom: 16 }}> </Text>
      <Text style={{ fontSize: 18, fontWeight: '600', color: '#333', marginBottom: 8 }}>
        Something went wrong
      </Text>
      <Text style={{ fontSize: 14, color: '#666', textAlign: 'center', marginBottom: 4 }}>
        {error?.message || 'Unknown error'}
      </Text>
      <ScrollView style={{ maxHeight: 300, width: '100%', marginTop: 12, backgroundColor:
'rgba(255, 255, 255, 0.5)', borderRadius: 8, padding: 12 }}>
        <Text style={{ fontSize: 10, color: '#999', fontFamily: 'monospace' }}>
          {error?.stack || 'No stack trace'}
        </Text>
      </ScrollView>
      <TouchableOpacity
        style={{ marginTop: 20, paddingHorizontal: 24, paddingVertical: 12, backgroundColor:
'rgba(255, 255, 255, 0.5)', borderRadius: 8 }}
        onPress={onReset}
      >
        <Text style={{ color: 'white', fontSize: 16, fontWeight: '600' }}>Retry</Text>
      </TouchableOpacity>
    </View>
  );
};
```

```
}

function AppContent() {
  const { token, setUser } = useAuthStore();
  const resolvedTheme = useThemeStore(s => s.resolvedTheme);
  const loadTheme = useThemeStore(s => s.loadTheme);
  const setSystemTheme = useThemeStore(s => s.setSystemTheme);
  const [isLoading, setIsLoading] = useState(true);
  const systemColorScheme = useColorScheme();
  const { t } = useI18n();

  useEffect(() => {
    initAnalytics().catch(() => {});
    initNotifications().catch(() => {});
  }, []);

  const isProcessingRef = useRef(false);
  const lastCheckedRef = useRef('');
  const consentRef = useRef(false);
  const checkClipboardRef = useRef<() => Promise<void> | null>(null);
  const CLIPBOARD_CONSENT_KEY = 'linkchest_clipboard_consent';

  useEffect(() => {
    setSystemTheme(systemColorScheme === 'dark');
  }, [systemColorScheme, setSystemTheme]);

  useEffect(() => {
    checkAuth();
  }, []);

  useEffect(() => {
    const handleUrl = (event: { url: string }) => {
      handleDeepLink(event.url);
    };

    Linking.getInitialURL().then((url) => {
      if (url) handleDeepLink(url);
    });

    const subscription = Linking.addEventListener('url', handleUrl);
    return () => { subscription.remove(); };
  }, []);

  const handleDeepLink = async (url: string) => {
```

```
if (!url) return;

if (url.startsWith('com.linkchest.app://')) {
  try {
    const parsed = new URL(url);
    const sharedUrl = parsed.searchParams.get('url') || '';
    const sharedTitle = parsed.searchParams.get('title') || '';

    if (!sharedUrl) {
      console.log('[DeepLink] No URL param in deep link');
      return;
    }

    let userSettings: any = {};
    try {
      const res = await api.get('/users/settings');
      userSettings = res.data?.data || {};
    } catch (err) {
      console.error('[DeepLink] Load settings failed:', err);
    }

    const shareMode = userSettings.shareMode || 'off';

    if (shareMode === 'off') {
      if (navigationRef.isReady()) {
        navigationRef.navigate('QuickAdd' as never, { mode: 'quickAdd', url: sharedUrl,
title: sharedTitle } as never);
      }
      return;
    }

    if (shareMode === 'quickSave') {
      if (!navigationRef.isReady()) return;
      try {
        const parseRes = await api.post('/collections/smart-parse', { input:
sharedUrl.trim() });
        const smartParsed = parseRes.data?.data;
        if (!smartParsed || !smartParsed.title) {
          Toast.show({
            type: 'error',
            text1: t('common.error'),
            text2: t('add.saveFailed'),
            visibilityTime: 4000,
          });
        }
      }
    }
  }
}
```

```
        return;
    }

    await api.post('/collections', {
        url: smartParsed.url || sharedUrl,
        title: smartParsed.title,
        coverImage: smartParsed.coverImage || undefined,
        platform: smartParsed.platform || undefined,
        tagIds: userSettings.defaultTagIds || [],
        listIds: userSettings.defaultListId ? [userSettings.defaultListId] : undefined,
    });

    Toast.show({
        type: 'success',
        text1: t('common.success'),
        text2: t('add.addedSuccess'),
        visibilityTime: 3000,
    });
} catch (err: any) {
    console.error('[DeepLink] Quick-save error:', err?.response?.data || err?.message
|| err);

    Toast.show({
        type: 'error',
        text1: t('common.error'),
        text2: t('add.saveFailed'),
        visibilityTime: 4000,
    });
}
return;
}
```

```
import { create } from 'zustand';
import * as SecureStore from 'expo-secure-store';
```

```
interface User {
    id: string;
    email: string | null;
    username: string | null;
    nickname: string | null;
    avatar: string | null;
    hasPassword: boolean;
}
```

```
interface AuthState {
```

```
    token: string | null;
    user: User | null;
    setToken: (token: string) => Promise<void>;
    setUser: (user: User) => void;
    logout: () => Promise<void>;
    loadToken: () => Promise<void>;
  }

export const useAuthStore = create<AuthState>((set) => ({
  token: null,
  user: null,

  setToken: async (token: string) => {
    await SecureStore.setItemAsync('linkchest_token', token);
    set({ token });
  },

  setUser: (user: User) => {
    set({ user });
  },

  logout: async () => {
    await SecureStore.deleteItemAsync('linkchest_token');
    set({ token: null, user: null });
  },

  loadToken: async () => {
    const token = await SecureStore.getItemAsync('linkchest_token');
    if (token) {
      set({ token });
    }
  },
}));

import { create } from 'zustand';
import AsyncStorage from '@react-native-async-storage/async-storage';

const THEME_KEY = 'linkchest-theme';

type Theme = 'light' | 'dark' | 'system';

interface ThemeState {
  theme: Theme;
  resolvedTheme: 'light' | 'dark';
```

```
colors: typeof lightColors;  
setTheme: (theme: Theme) => void;  
toggleTheme: () => void;  
loadTheme: () => Promise<void>;  
setSystemTheme: (isDark: boolean) => void;  
}
```

```
const lightColors = {  
  background: '#F7F5F0',  
  card: 'FFFFFF',  
  cardBorder: 'rgba(27,42,74,0.08)',  
  text: '#2D3142',  
  textSecondary: '#5A5A5A',  
  textTertiary: '#8A8175',  
  textMuted: '#B0A99E',  
  border: '#E8E4DC',  
  borderLight: '#F0EDE6',  
  primary: '#1B2A4A',  
  primaryBg: 'rgba(27,42,74,0.08)',  
  primaryText: '#1B2A4A',  
  input: 'FFFFFF',  
  inputBg: 'FFFFFF',  
  headerBg: '#1B2A4A',  
  headerText: '#F7F5F0',  
  secondaryBg: '#EDEAE3',  
  danger: '#B85C5C',  
  dangerBg: 'rgba(184,92,92,0.1)',  
  warning: '#C8956C',  
  success: '#5B8A72',  
  tagBg: 'rgba(27,42,74,0.06)',  
  tagText: '#1B2A4A',  
  listTagBg: 'rgba(200,149,108,0.12)',  
  listTagText: '#8B6914',  
  
  selectedBg: 'rgba(27,42,74,0.06)',  
  overlay: 'rgba(15,20,25,0.5)',  
  fabBg: '#1B2A4A',  
  modalBg: 'FFFFFF',  
  tabBg: '#1B2A4A',  
  tabActiveBg: '#C8956C',  
  tabText: '#8A8175',  
  tabActiveText: '#F7F5F0',  
  filterChipBg: '#EDEAE3',  
  filterChipActiveBg: '#1B2A4A',
```

```
filterChipText: '#5A5A5A',
filterChipActiveText: '#F7F5F0',
statDivider: '#E8E4DC',
sectionBg: '#FFFFFF',
menuBg: '#FFFFFF',
menuBorder: '#E8E4DC',
inactiveBg: 'rgba(184,92,92,0.1)',
inactiveText: '#B85C5C',
batchBg: 'rgba(27,42,74,0.06)',
batchText: '#1B2A4A',
surface: 'rgba(255, 255, 255, 0.8)',
surfaceElevated: 'rgba(255, 255, 255, 0.95)',
glow: 'rgba(200, 149, 108, 0.15)',
glowStrong: 'rgba(200, 149, 108, 0.35)',
shimmer: 'rgba(27, 42, 74, 0.05)',
};
```

```
const darkColors: typeof lightColors = {
  background: '#0F1419',
  card: '#1A1F2A',
  cardBorder: 'rgba(232,228,220,0.08)',
  text: '#E8E4DC',
  textSecondary: '#B0A99E',
  textTertiary: '#8A8175',
  textMuted: '#6B6560',
  border: '#2A303A',
  borderLight: '#1E242E',
  primary: '#C8956C',
  primaryBg: 'rgba(200,149,108,0.15)',
  primaryText: '#C8956C',
  input: '#1A1F2A',
  inputBg: '#1A1F2A',
  headerBg: '#0F1419',
  headerText: '#E8E4DC',
  secondaryBg: '#1E242E',
  danger: '#E07070',
  dangerBg: 'rgba(224,112,112,0.15)',

  warning: '#D4A574',
  success: '#6BA88A',
  tagBg: 'rgba(200,149,108,0.12)',
  tagText: '#D4A574',
  listTagBg: 'rgba(139,125,179,0.15)',
  listTagText: '#A599C8',
```

```
selectedBg: 'rgba(200,149,108,0.1)',
overlay: 'rgba(0,0,0,0.7)',
fabBg: '#C8956C',
modalBg: '#1A1F2A',
tabBg: '#0F1419',
tabActiveBg: '#C8956C',
tabText: '#8A8175',
tabActiveText: '#F7F5F0',
filterChipBg: '#1E242E',
filterChipActiveBg: '#C8956C',
filterChipText: '#B0A99E',
filterChipActiveText: '#0F1419',
statDivider: '#2A303A',
sectionBg: '#1A1F2A',
menuBg: '#1A1F2A',
menuBorder: '#2A303A',
inactiveBg: 'rgba(224,112,112,0.15)',
inactiveText: '#E07070',
batchBg: 'rgba(200,149,108,0.1)',
batchText: '#C8956C',
surface: 'rgba(26, 31, 42, 0.8)',
surfaceElevated: 'rgba(30, 36, 46, 0.95)',
glow: 'rgba(200, 149, 108, 0.25)',
glowStrong: 'rgba(200, 149, 108, 0.45)',
shimmer: 'rgba(232, 228, 220, 0.05)',
};

const getResolvedTheme = (theme: Theme, isSystemDark: boolean): 'light' | 'dark' => {
  if (theme === 'system') return isSystemDark ? 'dark' : 'light';
  return theme;
};

let systemIsDark = false;

export { lightColors, darkColors };

export const useThemeStore = create<ThemeState>((set, get) => ({
  theme: 'system',
  resolvedTheme: 'light',
  colors: lightColors,

  setTheme: (theme: Theme) => {
    AsyncStorage.setItem(THEME_KEY, theme);
    const resolved = getResolvedTheme(theme, systemIsDark);
```

```
    set({
      theme,
      resolvedTheme: resolved,
      colors: resolved === 'dark' ? darkColors : lightColors,
    });
  },

  toggleTheme: () => {
    const { theme } = get();
    const next = theme === 'light' ? 'dark' : theme === 'dark' ? 'system' : 'light';
    get().setTheme(next);
  },

  loadTheme: async () => {
    try {
      const saved = await AsyncStorage.getItem(THEME_KEY);
      if (saved === 'dark' || saved === 'light' || saved === 'system') {
        const resolved = getResolvedTheme(saved as Theme, systemIsDark);
        set({
          theme: saved as Theme,
          resolvedTheme: resolved,
          colors: resolved === 'dark' ? darkColors : lightColors,
        });
      }
    } catch (err) {
      console.warn('Failed to load theme:', err);
    }
  },

  setSystemTheme: (isDark: boolean) => {
    systemIsDark = isDark;
    const { theme } = get();
    if (theme === 'system') {
      const resolved = isDark ? 'dark' : 'light';
      set({
        resolvedTheme: resolved,
        colors: resolved === 'dark' ? darkColors : lightColors,
      });
    }
  },
});

import axios from 'axios';
import * as SecureStore from 'expo-secure-store';
```

```
import AsyncStorage from '@react-native-async-storage/async-storage';
import Toast from 'react-native-toast-message';
import { getErrorMessage, type SupportedLocale, isValidLocale } from '@linkchest/i18n';
import { getCachedTranslation } from './i18n';

const DEFAULT_API_URL = 'https://linkchest.net/api';
const API_URL_STORAGE_KEY = 'linkchest_api_url';

let currentBaseUrl = DEFAULT_API_URL;

export const api = axios.create({
  baseURL: DEFAULT_API_URL,
  timeout: 15000,
  headers: {
    'Content-Type': 'application/json',
  },
});

export async function initApiUrl() {
  try {
    const savedUrl = await SecureStore.getItemAsync(API_URL_STORAGE_KEY);
    if (savedUrl) {
      currentBaseUrl = savedUrl;
      api.defaults.baseURL = savedUrl;
    }
  } catch {
  }
}

export async function setApiUrl(url: string) {
  const normalized = url.endsWith('/api') ? url : url + '/api';
  currentBaseUrl = normalized;
  api.defaults.baseURL = normalized;
  await SecureStore.setItemAsync(API_URL_STORAGE_KEY, normalized);
}

export async function getApiUrl(): Promise<string> {
  try {
    const savedUrl = await SecureStore.getItemAsync(API_URL_STORAGE_KEY);
    return savedUrl || DEFAULT_API_URL;
  } catch {
    return DEFAULT_API_URL;
  }
}
```

```
export async function resetApiUrl() {
  currentBaseUrl = DEFAULT_API_URL;
  api.defaults.baseURL = DEFAULT_API_URL;
  await SecureStore.deleteItemAsync(API_URL_STORAGE_KEY);
}

export function getPublicBaseUrl(): string {
  return currentBaseUrl.replace('/api', '');
}

api.interceptors.request.use(
  async (config) => {
    const token = await SecureStore.getItemAsync('linkchest_token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => {
    return Promise.reject(error);
  }
);

api.interceptors.response.use(
  (response) => response,
  async (error) => {
    let locale: SupportedLocale = 'en';
    try {
      const saved = await AsyncStorage.getItem('linkchest-locale');
      if (saved && isValidLocale(saved)) locale = saved;
    } catch {}

    if (error.response?.data && typeof error.response.data === 'object') {
      const errCode = error.response.data.error;
      if (typeof errCode === 'string') {
        error.response.data.message = getErrorMessage(errCode, locale);
      }
    }

    const t = (key: string) => getCacheTranslation(locale)[key] || getCacheTranslation('en')[key] || key;

    if (error.response?.status === 401) {
```

```
await SecureStore.deleteItemAsync('linkchest_token');
const { useAuthStore } = require('../store/auth');
useAuthStore.setState({ token: null, user: null });
} else if (!error.response) {

const isTimeout = error.code === 'ECONNABORTED';
Toast.show({
  type: 'error',
  text1: isTimeout ? t('common.requestTimeout') : t('common.networkError'),
  text2: isTimeout ? t('common.tryLater') : t('common.checkNetwork'),
  visibilityTime: 3000,
});
} else if (error.response.status >= 500) {
  Toast.show({
    type: 'error',
    text1: t('common.serverBusy'),
    text2: t('common.tryLater'),
    visibilityTime: 3000,
  });
} else if (error.response.status === 403) {
  const errCode = error.response.data?.error || '';
  if (typeof errCode === 'string' && errCode.startsWith('QUOTA_EXCEEDED')) {
    Toast.show({
      type: 'error',
      text1: t('error.quotaExceeded'),
      text2: t('common.tryLater'),
      visibilityTime: 4000,
    });
  }
} else {
  Toast.show({
    type: 'error',
    text1: t('common.noAccess'),
    visibilityTime: 3000,
  });
}
} else if (error.response.status === 429) {
  const message = error.response.data?.message || t('common.rateLimited');
  Toast.show({
    type: 'error',
    text1: message,
    text2: t('common.tryLater'),
    visibilityTime: 4000,
  });
}
```

```
        return Promise.reject(error);
    }
};

// ===== API 辅助函数 =====

export async function uploadCover(imageData: string, originalName?: string) {
    const response = await api.post('/upload/cover', { imageData, originalName });
    return response.data?.data || response.data;
}

export async function getQuota() {
    const response = await api.get('/quota');
    return response.data?.data || response.data;
}

export async function recordShareView(shareId: string) {
    const response = await api.post(`/shares/${shareId}/view`);
    return response.data;
}

export async function importShare(shareId: string, syncTags = false) {
    const response = await api.post('/subscriptions/import', { shareId, syncTags });
    return response.data?.data || response.data;
}

export async function getMyTier() {
    const response = await api.get('/tiers/me');
    return response.data?.data || response.data;
}

export async function getTiers() {
    const response = await api.get('/tiers');
    return response.data?.data || response.data;
}

import React, { useState, useCallback, useEffect, useRef, useMemo, useLayoutEffect } from 'react';
import {
    View,
    Text,
    FlatList,
    TouchableOpacity,
    StyleSheet,
    RefreshControl,

```

```
    TextInput,
    Platform,
    Alert,
    Modal,
    ScrollView,
    ActivityIndicator,
    Animated,
    Linking,
  } from 'react-native';
import { useNavigation, useRoute, RouteProp, NavigationProp, useFocusEffect } from
 '@react-navigation/native';
import { Ionicons } from '@expo/vector-icons';
import { useQuery, useMutation, useQueryClient } from '../lib/react-query';
import Toast from 'react-native-toast-message';
import { api } from '../lib/api';
import { getPlatformName, getPlatformIcon, getPlatformColor, getPlatformConfig, buildDeepLink,
 PLATFORMS } from '../lib/platforms';

const PAGE_TYPE_ICONS: Record<string, string> = {
  home: 'home-outline',
  detail: 'document-text-outline',
  list: 'list-outline',
  search: 'search-outline',
  navigation: 'compass-outline',
  document: 'book-outline',
  download: 'download-outline',
  other: 'ellipsis-horizontal-outline',
};

const PAGE_TYPE_NAMES: Record<string, string> = {
  home: '主页',
  detail: '详情页',
  list: '列表页',
  search: '搜索页',
  navigation: '导航页',
  document: '文档页',
  download: '下载页',
  other: '其他',
};

function getPageTypeInfo(pageType: string | undefined) {
  const pt = pageType || 'detail';
  const icon = PAGE_TYPE_ICONS[pt] || PAGE_TYPE_ICONS.other;
  const name = PAGE_TYPE_NAMES[pt] || PAGE_TYPE_NAMES.other;
```

```

    return { icon, name };
  }
import { useThemeStore } from '../store/theme';
import { useI18n, getListDisplayName, getListPathDisplayName } from '../lib/i18n';
import { CollectionCardSkeleton, CollectionRowSkeleton } from
'../components/SkeletonComponents';
import { usePressableScale } from '../lib/animations';
import LazyImage from '../components/LazyImage';
import StarRating from '../components/StarRating';
import type { DisplayFieldKey, CollectionViews } from '../lib/collectionViewsStorage';
import { useCollectionViews } from '../lib/collectionViewsContext';

function getSortedFields(views: CollectionViews | undefined, mode: 'mobileGrid' | 'mobileList'):
[ key: DisplayFieldKey; enabled: boolean ][] {
  if (!views?.[mode]?.fields) {
    return [
      { key: 'cover', enabled: true },
      { key: 'title', enabled: true },
      { key: 'platform', enabled: true },
      { key: 'rating', enabled: true },
      { key: 'pageType', enabled: false },
      { key: 'tags', enabled: true },
      { key: 'lists', enabled: true },
      { key: 'note', enabled: true },
      { key: 'createdAt', enabled: false },
    ];
  }
  return [...views[mode].fields]
    .sort((a, b) => a.order - b.order)
    .map(f => ({ key: f.key, enabled: f.enabled }));
}

type RootStackParamList = {
  Main: undefined;
  Collections: { tagId?: string; listId?: string; tagName?: string; listName?: string };
  CollectionDetail: { id: string };
  AddCollection: { mode?: string };
  EditCollection: { id: string; mode?: string };
};

export type CollectionsRouteProp = RouteProp<RootStackParamList, 'Collections'>;
export type CollectionsNavigationProp = NavigationProp<RootStackParamList>;

interface Collection {

```

```
id: string;
title: string;
coverImage: string | null;
platform: string;
note: string | null;
tags?: { id: string; name: string }[];
lists?: { id: string; name: string }[];
url: string;
createdAt: string;
rating?: number | null;
pageType?: string;
}
```

```
interface List {
  id: string;
  name: string;
}
```

```
function PlatformDot({ platform }: { platform: string }) {
  const color = getPlatformColor(platform);
  const name = getPlatformName(platform);
  const initial = name ? name.charAt(0) : '?';
  return (
    <View style={{ width: 28, height: 28, borderRadius: 14, backgroundColor: color,
justifyContent: 'center', alignItems: 'center' }}>
      <Text style={{ fontSize: 12, fontWeight: 'bold', color: '#FFFFFF' }}>{initial}</Text>
    </View>
  );
}
```

```
interface CollectionItemProps {
  item: Collection;
  colors: any;
  editMode: boolean;
  selected: boolean;
  onPress: () => void;
  onImagePress: () => void;
  onLongPress: () => void;
  t?: any;
  enabledFields?: Set<string>;
}
```

```
const CardItem = React.memo(function CardItem({ item, colors, editMode, selected, onPress,
onImagePress, onLongPress, t, enabledFields }: CollectionItemProps) {
```

```

const { scaleValue, onPressIn, onPressOut } = usePressableScale(1, 0.97);
if (item.id === '__placeholder__') {
  return <View style={{ flex: 1, margin: 4 }} />;
}
const ef = enabledFields || new Set(['cover', 'title', 'platform', 'rating', 'tags', 'lists', 'note']);

return (
  <Animated.View style={{ flex: 1, margin: 4, transform: [{ scale: scaleValue }] }}>
    <View
      style={{ backgroundColor: colors.card, borderRadius: 12, overflow: 'hidden',
borderWidth: editMode && selected ? 2 : 0, borderColor: editMode && selected ?
colors.primary : 'transparent', shadowColor: '#000', shadowOffset: { width: 0, height: 2 },
shadowOpacity: 0.06, shadowRadius: 8, elevation: 3 }}
    >
      {editMode && (
        <TouchableOpacity
          style={{ position: 'absolute', top: 8, left: 8, zIndex: 10, backgroundColor:
colors.card + 'CC', borderRadius: 12 }}
          onPress={onPress}
        >
          <Icons name={selected ? 'checkmark-circle' : 'ellipse-outline'} size={24}
color={selected ? colors.primary : colors.textTertiary} />
        </TouchableOpacity>
      )}
      {ef.has('cover') && (
        <TouchableOpacity      onPress={onImagePress}      onLongPress={onLongPress}
onPressIn={onPressIn} onPressOut={onPressOut} activeOpacity={0.9}>
          <LazyImage uri={item.coverImage} style={{ width: '100%', aspectRatio: 1 }}
fallbackPlatform={item.platform} />
        </TouchableOpacity>
      )}
      <View style={{ padding: 10, gap: 4 }}>
        <TouchableOpacity      onPress={onPress}      onLongPress={onLongPress}
onPressIn={onPressIn} onPressOut={onPressOut} activeOpacity={0.9} style={{ flex: 1 }}>
          {ef.has('title') && (
            <Text style={{ fontSize: 12, fontWeight: '600', color: colors.text, lineHeight: 17 }}
numberOfLines={2}>{item.title}</Text>
          )}
          {(ef.has('platform') || ef.has('rating')) && (
            <View style={{ flexDirection: 'row', alignItems: 'center', gap: 3, marginTop: 3 }}>
              {ef.has('platform') && (
                <>
                  <Icons name={getPlatformIcon(item.platform) as any} size={11}
color={getPlatformColor(item.platform)} />

```

```

        <Text style={{ fontSize: 10, color:
colors.textSecondary }}>{getPlatformName(item.platform)}</Text>
      </>
    )}
    {ef.has('rating') && item.rating !== undefined && item.rating !== null && (
      <StarRating value={item.rating} readonly size={11} />
    )}
  </View>
)}

{ef.has('note') && item.note && (
  <Text style={{ fontSize: 10, color: colors.textTertiary, marginTop: 2 }}
numberOfLines={1}>{item.note}</Text>
)}
</TouchableOpacity>
<View style={{ flexDirection: 'row', flexWrap: 'wrap', gap: 3 }}>
  {ef.has('pageType') && (() => {
    const ptInfo = getPageTypeInfo(item.pageType);
    return (
      <View style={{ backgroundColor: colors.primaryBg, paddingHorizontal: 4,
paddingVertical: 1.5, borderRadius: 3, flexDirection: 'row', alignItems: 'center', gap: 2 }}>
        <Ionicons name={ptInfo.icon as any} size={8} color={colors.primary} />
        <Text style={{ fontSize: 8, color: colors.primary }}>{ptInfo.name}</Text>
      </View>
    );
  })()}
  {ef.has('tags') && item.tags?.slice(0, 2).map((tag) => (
    <View key={tag.id} style={{ backgroundColor: colors.tagBg, paddingHorizontal: 4,
paddingVertical: 1.5, borderRadius: 3 }}>
      <Text style={{ fontSize: 8, color: colors.tagText }}>#{tag.name}</Text>
    </View>
  ))}
  {ef.has('lists') && item.lists?.slice(0, 1).map((list) => (
    <View key={list.id} style={{ backgroundColor: colors.listTagBg, paddingHorizontal:
4, paddingVertical: 1.5, borderRadius: 3, flexDirection: 'row', alignItems: 'center', gap: 2 }}>
      <Ionicons name="folder-outline" size={7} color={colors.listTagText} />
      <Text style={{ fontSize: 8, color: colors.listTagText }}>{getListDisplayName(list,
t)}</Text>
    </View>
  ))}
  {ef.has('createdAt') && item.createdAt && (
    <Text style={{ fontSize: 8, color: colors.textTertiary }}>{new
Date(item.createdAt).toLocaleDateString()}</Text>
  )}

```

```
        </View>
      </View>
    </View>
  </Animated.View>
);
});
```

```
const GridItem = React.memo(function GridItem({ item, colors, editMode, selected, onPress,
onImagePress, onLongPress, t, enabledFields }: CollectionItemProps) {
  const { scaleValue, onPressIn, onPressOut } = usePressableScale(1, 0.98);
  const ef = enabledFields || new Set(['cover', 'title', 'platform', 'rating', 'tags', 'lists', 'note']);
```

```
import React, { useState, useCallback, useRef, useEffect } from 'react';
import {
  View,
  Text,
  TextInput,
  TouchableOpacity,
  KeyboardAvoidingView,
  Platform,
  Alert,
  ActivityIndicator,
  ScrollView,
  Modal,
  StyleSheet,
  Animated,
  Image,
} from 'react-native';
import { useNavigation } from '@react-navigation/native';
import { api } from '../lib/api';
import { useAuthStore } from '../store/auth';
import { useThemeStore } from '../store/theme';
import { useI18n, type SupportedLocale } from '../lib/i18n';
import { ErrorCodeToI18nKey, AuthErrorCodes } from '../lib/errorCodes';
import { Ionicons } from '@expo/vector-icons';
import { usePressableScale } from '../lib/animations';
import * as Google from 'expo-auth-session/providers/google';
import * as WebBrowser from 'expo-web-browser';
import Constants from 'expo-constants';
```

```
WebBrowser.maybeCompleteAuthSession();
```

```
type AccountType = 'email';
```

```
function detectAccountType(value: string): 'email' | 'unknown' {
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (emailRegex.test(value)) return 'email';
  return 'unknown';
}

function getErrorMessage(errorCode: string, t: (key: string) => string): string {
  const i18nKey = ErrorCodeToI18nKey[errorCode as keyof typeof ErrorCodeToI18nKey];
  if (i18nKey) {
    return t(i18nKey);
  }
  return t('error.unknown');
}

const AnimatedTouchableOpacity = Animated.createAnimatedComponent(TouchableOpacity);

function ScaleButton({ children, style, onPress, disabled, activeOpacity = 0.85 }: { children:
React.ReactNode; style?: any; onPress?: () => void; disabled?: boolean; activeOpacity?: number })
{
  const { scaleValue, onPressIn, onPressOut } = usePressableScale(1, 0.97);
  return (
    <AnimatedTouchableOpacity
      onPress={onPress}
      onPressIn={onPressIn}
      onPressOut={onPressOut}
      disabled={disabled}
      activeOpacity={activeOpacity}
      style={[{ transform: [{ scale: scaleValue }] }, style]}
    >
      {children}
    </AnimatedTouchableOpacity>
  );
}

export default function LoginScreen() {
  const [lang, setLang] = useState<SupportedLocale>('en');
  const colors = useThemeStore(s => s.colors);
  const { t, locale, setLocale } = useI18n();
  const navigation = useNavigation();

  useEffect(() => {
    setLang(locale);
  }, [locale]);
}
```

```
const [account, setAccount] = useState("");
const [password, setPassword] = useState("");
const [showPassword, setShowPassword] = useState(false);
const [loading, setLoading] = useState(false);

const [showRegister, setShowRegister] = useState(false);
const registerType: AccountType = 'email';
const [regAccount, setRegAccount] = useState("");
const [regCode, setRegCode] = useState("");
const [regPassword, setRegPassword] = useState("");
const [regConfirmPassword, setRegConfirmPassword] = useState("");
const [regReferralCode, setRegReferralCode] = useState("");
const [regLoading, setRegLoading] = useState(false);
const [countdown, setCountdown] = useState(0);

const [showForgot, setShowForgot] = useState(false);
const [forgotAccount, setForgotAccount] = useState("");
const [forgotCode, setForgotCode] = useState("");
const [forgotNewPassword, setForgotNewPassword] = useState("");
const [forgotConfirmPassword, setForgotConfirmPassword] = useState("");
const [forgotLoading, setForgotLoading] = useState(false);

const [showSetPassword, setShowSetPassword] = useState(false);
const [setPwd, setSetPwd] = useState("");
const [setPwdConfirm, setSetPwdConfirm] = useState("");
const [setPwdLoading, setSetPwdLoading] = useState(false);

const { setToken, setUser } = useAuthStore();

const googleClientId = Constants.expoConfig?.extra?.googleClientId as string | undefined;
const [googleRequest, googleResponse, promptGoogleAsync] = Google.useAuthRequest({
  expoClientId: googleClientId,
  androidClientId: googleClientId,
  scopes: ['openid', 'profile', 'email'],
});

useEffect(() => {
  if (googleResponse?.type === 'success') {
    const { id_token } = googleResponse.params;
    if (id_token) {
      handleGoogleLogin(id_token);
    }
  } else if (googleResponse?.type === 'error') {
    Alert.alert(t('common.error'), t('login.googleLoginFailed'));
  }
});
```

```
    }  
  }, [googleResponse]);  
  
const handleGoogleLogin = async (credential: string) => {  
  try {  
    setLoading(true);  
    const response = await api.post('/auth/google', { credential, lang });  
    const { token, user } = response.data;  
    await setToken(token);  
    setUser(user);  
    if (!user.hasPassword) {  
      setSetPwd("");  
      setSetPwdConfirm("");  
      setShowSetPassword(true);  
    } else {  
  
      navigation.replace('Main' as never);  
    }  
  } catch (error: any) {  
    const errorCode = error.response?.data?.error || AuthErrorCodes.SERVER_ERROR;  
    Alert.alert(t('common.error'), getErrorMessage(errorCode, t));  
  } finally {  
    setLoading(false);  
  }  
};  
  
const countdownTimerRef = useRef<ReturnType<typeof setInterval> | null>(null);  
  
useEffect(() => {  
  return () => {  
    if (countdownTimerRef.current) {  
      clearInterval(countdownTimerRef.current);  
    }  
  };  
}, []);  
  
const handleLanguageChange = useCallback((newLang: SupportedLocale) => {  
  setLang(newLang);  
  setLocale(newLang);  
}, [setLocale]);  
  
const sendCode = async (targetAccount?: string) => {  
  const target = targetAccount || regAccount;  
  if (!/^[^\\s@]+@[^\\s@]+\\.^[^\\s@]+$/\\.test(target)) {
```

```
    Alert.alert(t('common.hint'), t('error.invalidEmailFormat'));
    return;
  }

  try {
    setRegLoading(true);
    const response = await api.post('/auth/send-code', {
      email: target,
      lang
    });
    const realCode = response.data?.code;
    if (realCode) {
      Alert.alert(t('account.verificationCode'), t('login.yourCodeIs', { code: realCode }));
    } else {
      Alert.alert(t('common.hint'), t('login.codeSent'));
    }

    setCountdown(60);
    if (countdownTimerRef.current) clearInterval(countdownTimerRef.current);
    countdownTimerRef.current = setInterval(() => {
      setCountdown((prev) => {
        if (prev <= 1) {
          if (countdownTimerRef.current) clearInterval(countdownTimerRef.current);

          return 0;
        }
        return prev - 1;
      });
    }, 1000);
  } catch (error: any) {
    const errorCode = error.response?.data?.error || AuthErrorCodes.SERVER_ERROR;
    Alert.alert(t('common.error'), getErrorMessage(errorCode, t));
  } finally {
    setRegLoading(false);
  }
};

const handleLogin = async () => {
  if (!account || !password) {
    Alert.alert(t('common.hint'), t('login.enterAccountAndPassword'));
    return;
  }

  const accountType = detectAccountType(account);
```

```
    if (accountType === 'unknown') {
      Alert.alert(t('common.hint'), t('login.invalidAccountFormat'));
      return;
    }

    try {
      setLoading(true);
      const response = await api.post('/auth/login-email', { email: account, password, lang });
      const { token, user } = response.data;

      await setToken(token);
      setUser(user);
      navigation.replace('Main' as never);
    } catch (error: any) {
      const errorCode = error.response?.data?.error || AuthErrorCodes.SERVER_ERROR;
      Alert.alert(t('common.error'), getErrorMessage(errorCode, t));
    } finally {
      setLoading(false);
    }
  };

import React, { useState } from 'react';
import {
  View,
  Text,
  StyleSheet,
  TouchableOpacity,
  Alert,
  ScrollView,
  Share,
  Linking,
  Image,
  Modal,
} from 'react-native';
import { useNavigation, useFocusEffect } from '@react-navigation/native';
import { Ionicons } from '@expo/vector-icons';
import { useQuery } from '../lib/react-query';
import { useAuthStore } from '../store/auth';
import { useThemeStore } from '../store/theme';
import { api } from '../lib/api';
import { useI18n } from '../lib/i18n';

export default function ProfileScreen() {
  const navigation = useNavigation();
```

```
const { user, setUser, logout } = useAuthStore();
const { theme, resolvedTheme, toggleTheme, colors } = useThemeStore();
const { t, locale, setLocale } = useI18n();
const [showLangModal, setShowLangModal] = useState(false);

const { data: stats, refetch: refetchStats } = useQuery({
  queryKey: ['stats'],
  queryFn: async () => {
    try {
      const overviewRes = await api.get('/stats/overview');
      const d = overviewRes.data.data;
      return {
        collections: d?.collectionCount || 0,
        lists: d?.listCount || 0,
        shareCount: d?.shareCount || 0,
        tags: d?.tagCount || 0,
        shareViewCount: d?.shareViewCount || 0,
      };
    } catch {
      return { collections: 0, lists: 0, shareCount: 0, tags: 0, shareViewCount: 0 };
    }
  },
});

useFocusEffect(
  React.useCallback(() => {
    api.get('/auth/me').then((res: any) => {
      const userData = res.data.data || res.data;
      setUser(userData);
    }).catch(() => {});
    refetchStats();
  }, [refetchStats])
);

const handleLogout = () => {
  Alert.alert(t('profile.logoutConfirm'), '', [
    { text: t('common.cancel'), style: 'cancel' },
    {
      text: t('profile.logoutBtn'),
      style: 'destructive',
      onPress: () => {
        logout();
        navigation.reset({
```

```
        index: 0,
        routes: [{ name: 'Login' as any }],
    });
},
},
]);
};

const dataManagementItems = [
    {
        icon: 'download-outline',
        title: t('export.title'),
        onPress: () => navigation.navigate('Export' as any),
    },
    {
        icon: 'stats-chart-outline',
        title: t('profile.platformStats'),
        onPress: () => navigation.navigate('PlatformStats' as any),
    },
    {
        icon: 'scan-outline',
        title: t('settings.duplicateDetect'),
        onPress: () => navigation.navigate('DuplicateDetect' as any),
    },
    {
        icon: 'cloud-upload-outline',
        title: t('settings.autoBackup'),
        onPress: () => navigation.navigate('AutoBackup' as any),
    },
    {
        icon: 'trash-outline',
        title: t('collection.trash.title'),
        onPress: () => navigation.navigate('Trash' as any),
    },
];

const systemSettingsItems = [
    {
        icon: 'flash-outline',
        title: t('profile.quickSaveSettings'),
        onPress: () => navigation.navigate('QuickSaveSettings' as any),
    },
    {
```

```

        icon: 'grid-outline',
        title: '收藏展示设置',
        onPress: () => navigation.navigate('CollectionViewConfig' as any),
      },
      {
        icon: theme === 'dark' ? 'moon-outline' : theme === 'system' ? 'phone-portrait-outline' :
'sunny-outline',
        title: theme === 'dark' ? t('profile.lightMode') : theme === 'system' ?
t('profile.followSystem') : t('profile.darkMode'),
        value: theme === 'system' ? (resolvedTheme === 'dark' ? t('profile.dark') : t('profile.light')) :
undefined,
        onPress: toggleTheme,
      },
      {
        icon: 'language-outline',
        title: t('profile.language'),
        value: locale === 'zh' ? t('profile.languageZh')
          : locale === 'ja' ? t('profile.languageJa')
          : locale === 'ko' ? t('profile.languageKo')
          : locale === 'fr' ? t('profile.languageFr')
          : locale === 'de' ? t('profile.languageDe')
          : t('profile.languageEn'),
        onPress: () => setShowLangModal(true),
      },
    ],
  ];

```

```

const otherItems = [
  {
    icon: 'document-text-outline',
    title: t('terms.titleAndPrivacy'),
    onPress: () => navigation.navigate('Terms' as any, { tab: 'terms' }),
  },
  {
    icon: 'refresh-outline',
    title: t('profile.checkUpdate'),
  },
];

```

```

import React, { useEffect, useState, useCallback, useRef, useMemo } from 'react';
import {
  View,
  Text,
  TextInput,
  TouchableOpacity,
  ScrollView,
  Alert,

```

```
    ActivityIndicator,  
    Modal,  
  } from 'react-native';  
import { useRoute, useNavigation, useFocusEffect, RouteProp } from '@react-navigation/native';  
import { Ionicons } from '@expo/vector-icons';  
import { useQuery, useMutation, useQueryClient } from '../lib/react-query';  
import { api } from '../lib/api';  
import { getPlatformName, getPlatformColor, getPlatformIcon } from '../lib/platforms';  
import { useThemeStore } from '../store/theme';  
import { useI18n, getListDisplayName, getListPathDisplayName } from '../lib/i18n';  
import { logEvent } from '../lib/analytics';  
import CoverEditor from '../components/CoverEditor';  
import StarRating from '../components/StarRating';  
import { PAGE_TYPES, DEFAULT_PAGE_TYPE, getPageTypeConfig } from '../lib/pageTypes';
```

```
type CollectionFormMode = 'quickAdd' | 'add' | 'edit';
```

```
type CollectionFormRouteProp = RouteProp<{  
  CollectionForm: {  
    mode: CollectionFormMode;  
    id?: string;  
    url?: string;  
    title?: string;  
    tagId?: string;  
    listId?: string;  
  };  
}, 'CollectionForm'>;
```

```
interface Tag {  
  id: string;  
  name: string;  
  collectionCount?: number;  
}
```

```
interface ListItem {  
  id: string;  
  name: string;  
  parentId: string | null;  
  collectionCount: number;  
  totalCollectionCount?: number;  
  isDefault?: boolean;  
  depth?: number;  
  path?: { id: string; name: string; isDefault?: boolean }[];  
  pathName?: string | null;
```

```

        <Ionicons      name="pricetag-outline"      size={16}      color={colors.textTertiary}
style={{ marginTop: 2 }} />
        <View style={{ flex: 1, flexDirection: 'row', alignItems: 'center', gap: 4, flexWrap:
'wrap' }}>
            {(collection.tags?.length ?? 0) > 0 ? (
                <>
                    {(collection.tags || []).slice(0, 5).map((tag) => (
                        <View      key={tag.id}      style={{      backgroundColor:      colors.tagBg,
paddingHorizontal: 6, paddingVertical: 2, borderRadius: 4 }}>
                            <Text style={{ color: colors.tagText, fontSize: 12 }}>#{tag.name}</Text>
                        </View>
                    ))}
                    {(collection.tags || []).length > 5 && (
                        <Text style={{ fontSize: 12, color: colors.textTertiary }}>+{(collection.tags ||
[]).length - 5}</Text>
                    )}
                </>
            ) : (
                <Text style={{ fontSize: 12, color: colors.textTertiary }}>-</Text>
            )}
        </View>
    </View>

```

```

import React, { useState, useCallback } from 'react';
import {
    View,
    Text,
    FlatList,
    TouchableOpacity,
    StyleSheet,
    Alert,
    TextInput,
    Modal,
    RefreshControl,
    ScrollView,
    ActivityIndicator,
    Linking,
} from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import { useQuery, useMutation, useQueryClient } from '../lib/react-query';
import { useNavigation, NavigationProp, useFocusEffect } from '@react-navigation/native';
import { api } from '../lib/api';

```

```
import { useThemeStore } from '../store/theme';
import { getPlatformConfig, buildDeepLink } from '../lib/platforms';
import { useI18n } from '../lib/i18n';
import { TagManageSkeleton } from '../components/SkeletonComponents';
import LazyImage from '../components/LazyImage';

interface Tag {
  id: string;
  name: string;
  collectionCount: number;
}

interface Collection {
  id: string;
  title: string;
  coverImage: string | null;
  platform: string;
  url: string;
}

type RootStackParamList = {
  Main: undefined;
  Collections: { tagId?: string; tagName?: string; listId?: string; listName?: string };
  CollectionDetail: { id: string };
};

export default function TagManageScreen() {
  const queryClient = useQueryClient();
  const navigation = useNavigation<NavigationProp<RootStackParamList>>();
  const colors = useThemeStore(s => s.colors);
  const { t } = useI18n();
  const [modalVisible, setModalVisible] = useState(false);
  const [editingTag, setEditingTag] = useState<Tag | null>(null);

  const [tagName, setTagName] = useState('');
  const [refreshing, setRefreshing] = useState(false);
  const [detailTag, setDetailTag] = useState<{ id: string; name: string; collectionCount: number } | null>(null);
  const [detailExpanded, setDetailExpanded] = useState(true);

  const { data: detailCollections, isLoading: detailLoading } = useQuery({
    queryKey: ['collections', 'tagDetail', detailTag?.id],
    queryFn: async () => {
      if (!detailTag) return [] as Collection[];
    }
  });
```

```

        const response = await api.get(`/collections?tagId=${detailTag.id}&limit=20`);
        return (Array.isArray(response.data) ? response.data
            : (response.data?.data || response.data?.collections || [])) as Collection[];
    },
    enabled: !!detailTag,
});

const { data: tags, isLoading, refetch: refetchTags } = useQuery({
    queryKey: ['tags'],
    queryFn: async () => {
        const response = await api.get('/tags');
        return (response.data.data || response.data) as Tag[];
    },
});

const onRefresh = async () => { setRefreshing(true); await refetchTags(); setRefreshing(false); };

const openUrl = async (url: string, platform?: string) => {
    const platformConfig = platform ? getPlatformConfig(platform) : undefined;
    if (platformConfig?.appSchemes && platformConfig.appSchemes.length > 0) {
        const deepLink = buildDeepLink(url, platform!);
        if (deepLink) {
            try {
                const supported = await Linking.canOpenURL(deepLink);
                if (supported) {
                    await Linking.openURL(deepLink);
                    return;
                }
            } catch {}
        }
    }
    try {
        await Linking.openURL(url);
    } catch {
        Alert.alert(t('common.hint'), t('share.view.openFailed'));
    }
};

useFocusEffect(useCallback(() => { refetchTags(); }, [refetchTags]));

const createMutation = useMutation({
    mutationFn: (name: string) => api.post('/tags', { name }),
    onSuccess: (res: any) => {
        queryClient.invalidateQueries({ queryKey: ['tags'] });
        setModalVisible(false);
    }
});

```

```
    setTagName("");
    const data = res.data?.data || res.data;
    if (data?.renamed) { Alert.alert(t('common.hint'), t('tag.tagNameExistsAuto',
{ originalName: data.originalName, name: data.name })); }
  },
  onError: (error: any) => { Alert.alert(t('common.error'), error.response?.data?.message ||
error.response?.data?.error || t('tag.createFailed')); },
});

const updateMutation = useMutation({
  mutationFn: ({ id, name }: { id: string; name: string }) => api.put(`/tags/${id}`, { name }),
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['tags'] });
    setModalVisible(false);
    setEditingTag(null);
    setTagName("");
  },
  onError: (error: any) => { Alert.alert(t('common.error'), error.response?.data?.message ||
error.response?.data?.error || t('tag.updateFailed')); },
});

const deleteMutation = useMutation({
  mutationFn: (id: string) => api.delete(`/tags/${id}`),
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['tags'] });
    queryClient.invalidateQueries({ queryKey: ['collections'] });
  },
});

const handleSave = () => {
  if (!tagName.trim()) { Alert.alert(t('common.hint'), t('tag.enterTagName')); return; }
  if (editingTag) { updateMutation.mutate({ id: editingTag.id, name: tagName.trim() }); }
  else {
    const exists = tags?.some((tg: any) => tg.name === tagName.trim());
    if (exists) {
      Alert.alert(t('tag.nameDuplicate'), t('tag.nameExists', { name: tagName.trim() }), [
        { text: t('common.cancel'), style: 'cancel' },
        { text: t('tag.continueCreate'), onPress: () =>
createMutation.mutate(tagName.trim()) },
      ]);
    } else {
      createMutation.mutate(tagName.trim());
    }
  }
}
```

```
};

const handleEdit = (tag: Tag) => { setEditingTag(tag); setTagName(tag.name);
setModalVisible(true); };

import React, { useState, useEffect, useMemo, useRef } from 'react';
import { useNavigation } from '@react-navigation/native';
import {
  View,
  Text,
  StyleSheet,
  TouchableOpacity,
  Alert,
  ScrollView,
  TextInput,
  Modal,
  KeyboardAvoidingView,
  Platform,
  Image,
  ActivityIndicator,
  Linking,
} from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import * as ImagePicker from 'expo-image-picker';
import * as FileSystem from 'expo-file-system';
import * as Clipboard from 'expo-clipboard';
import { useQuery, useMutation, useQueryClient } from '../lib/react-query';
import { useAuthStore } from '../store/auth';
import { api, setApiUrl, resetApiUrl, getApiUrl } from '../lib/api';
import { useThemeStore } from '../store/theme';
import { useI18n } from '../lib/i18n';

export default function AccountSettingsScreen() {
  const navigation = useNavigation();
  const { user, setUser } = useAuthStore();
  const queryClient = useQueryClient();
  const colors = useThemeStore(s => s.colors);
  const { t, locale } = useI18n();

  const [usernameModal, setUsernameModal] = useState(false);
  const [usernameValue, setUsernameValue] = useState(user?.username || '');

  const [passwordModal, setPasswordModal] = useState(false);
  const [oldPassword, setOldPassword] = useState('');
```

```
const [newPassword, setNewPassword] = useState("");
const [confirmPassword, setConfirmPassword] = useState("");

const [emailModal, setEmailModal] = useState(false);
const [emailValue, setEmailValue] = useState(user?.email || '');
const [emailCode, setEmailCode] = useState("");
const [emailCountdown, setEmailCountdown] = useState(0);
const [emailSending, setEmailSending] = useState(false);
const emailTimerRef = useRef<ReturnType<typeof setInterval> | null>(null);

const [serverModal, setServerModal] = useState(false);
const [serverValue, setServerValue] = useState("");
const [currentServer, setCurrentServer] = useState("");
const [avatarUploading, setAvatarUploading] = useState(false);
const [showCoverPicker, setShowCoverPicker] = useState(false);
const [tierData, setTierData] = useState<any>(null);
const [tierLoading, setTierLoading] = useState(true);

const [referralCode, setReferralCode] = useState<string | null>(null);
const [referralStats, setReferralStats] = useState<any>(null);
const [referralLoading, setReferralLoading] = useState(false);
const [referralError, setReferralError] = useState(false);

useEffect(() => {
  loadTier();
}, []);

useEffect(() => {
  loadReferralData();
}, []);

async function loadTier() {
  try {
    setTierLoading(true);
    const res = await api.get('/tiers/me');
    setTierData(res.data?.data || res.data);
  } catch {
  } finally {
    setTierLoading(false);
  }
}

async function loadReferralData() {
  try {
```

```

        setReferralLoading(true);
        setReferralError(false);
        const codeRes = await api.post('/referrals/code');
        setReferralCode(codeRes.data?.data?.code || null);
        const statsRes = await api.get('/referrals/stats');
        setReferralStats(statsRes.data?.data || null);
      } catch {
        setReferralError(true);
      } finally {
        setReferralLoading(false);
      }
    }
  }

function formatPrice(pricing: any): string {
  if (!pricing || typeof pricing !== 'object') return t('tier.free');
  const monthly = pricing.monthly;
  if (monthly && typeof monthly === 'object' && 'usd' in monthly) {
    const usd = monthly.usd;
    if (typeof usd === 'number' && usd > 0) {
      return `${(usd / 100).toFixed(2)}${t('tier.perMonth')}`;
    }
  }
  return t('tier.free');
}

const tierColor = (k: string) => ({ medium: '#8A8175', heavy: '#1B2A4A', super: '#C8956C' }[k]
|| colors.primary);

React.useEffect(() => {
  getApiUrl().then(url => setCurrentServer(url.replace('/api', '')));
}, []);

const { data: systemCoversData } = useQuery({
  queryKey: ['systemCovers'],
  queryFn: async () => {
    const response = await api.get('/upload/system-covers');
    return response.data;
  },
  enabled: showCoverPicker,
});

const { data: coverLibraryData } = useQuery({
  queryKey: ['coverLibrary'],
  queryFn: async () => {

```

```

        const response = await api.get('/upload/covers?limit=50');
        return response.data;
      },
      enabled: showCoverPicker,
    });

const profileMutation = useMutation({
  mutationFn: (data: any) => api.patch('/auth/profile', data),
  onSuccess: () => {
    api.get('/auth/me').then((res: any) => {
      setUser(res.data.data || res.data);
    }).catch(() => {});
    queryClient.invalidateQueries({ queryKey: ['user'] });
  },
  onError: (error: any) => {
    Alert.alert(t('common.error'), error.response?.data?.message ||
error.response?.data?.error || t('common.operationFailed'));
  },
});

import React, { useState } from 'react';
import {
  View,
  Text,
  TouchableOpacity,
  ScrollView,
  Alert,
  ActivityIndicator,
  TextInput,
  Share,
} from 'react-native';
import { useNavigation } from '@react-navigation/native';
import { Ionicons } from '@expo/vector-icons';
import { useQuery, useMutation, useQueryClient } from '../lib/react-query';
import Toast from 'react-native-toast-message';
import { api } from '../lib/api';
import { getPlatformIcon, getPlatformName } from '../lib/platforms';
import { useThemeStore } from '../store/theme';
import { useI18n, getListPathDisplayName } from '../lib/i18n';
import { logEvent } from '../lib/analytics';

interface List {
  id: string;
  name: string;

```

```
    parentId: string | null;  
    collectionCount: number;  
    totalCollectionCount?: number;  
    isDefault?: boolean;  
    depth?: number;  
    path?: { id: string; name: string; isDefault?: boolean }[];  
    pathName?: string | null;  
  }
```

```
interface Tag {  
  id: string;  
  name: string;  
  collectionCount: number;  
}
```

```
interface Collection {  
  id: string;  
  title: string;  
  coverImage: string | null;  
  platform: string;  
}
```

```
type ShareType = 'ALL' | 'COLLECTION' | 'LIST' | 'TAG';  
type ExpiresIn = '1h' | '24h' | '1w' | 'never';
```

```
export default function CreateShareScreen() {  
  const navigation = useNavigation();  
  const colors = useThemeStore(s => s.colors);  
  const { t } = useI18n();  
  
  const queryClient = useQueryClient();  
  const [shareType, setShareType] = useState<ShareType>('ALL');  
  const [selectedLists, setSelectedLists] = useState<string[]>([]);  
  const [selectedTags, setSelectedTags] = useState<string[]>([]);  
  const [selectedCollections, setSelectedCollections] = useState<string[]>([]);  
  const [searchQuery, setSearchQuery] = useState('');  
  const [generatedLink, setGeneratedLink] = useState<string | null>(null);  
  
  const [shareTitle, setShareTitle] = useState('');  
  const [expiresIn, setExpiresIn] = useState<ExpiresIn>('never');  
  const [password, setPassword] = useState('');  
  const [description, setDescription] = useState('');  
  const [showAdvanced, setShowAdvanced] = useState(false);
```

```
const { data: lists } = useQuery({
  queryKey: ['lists'],
  queryFn: async () => {
    const response = await api.get('/lists/flat');
    return (response.data.data || response.data) as List[];
  },
});

const { data: tags } = useQuery({
  queryKey: ['tags'],
  queryFn: async () => {
    const response = await api.get('/tags');
    return (response.data.data || response.data) as Tag[];
  },
});

const { data: collections } = useQuery({
  queryKey: ['collections'],
  queryFn: async () => {
    const response = await api.get('/collections');
    return (response.data.data || response.data) as Collection[];
  },
});

const [returnedPassword, setReturnedPassword] = useState<string | null>(null);

const createMutation = useMutation({
  mutationFn: (data: any) => api.post('/shares', data),
  onSuccess: (response) => {
    queryClient.invalidateQueries({ queryKey: ['shares'] });
    logEvent('create_share', { type: shareType });
    const shareUrl = response.data.shareUrl;
    setGeneratedLink(shareUrl);

    if (response.data.password) {
      setReturnedPassword(response.data.password);
    }
    Toast.show({ type: 'success', text1: t('share.create.linkGenerated') });
  },
  onError: (error: any) => {
    Alert.alert(t('common.error'), error.response?.data?.message ||
error.response?.data?.error || t('share.create.createFailed'));
  },
});
```

```
const handleCreate = () => {
  let defaultTitle = '';
  const data: any = { type: shareType, expiresIn, description: description.trim() || undefined };

  if (password.trim()) {
    data.password = password.trim();
  }

  switch (shareType) {
    case 'ALL':
      defaultTitle = t('share.typeAll');
      break;
    case 'COLLECTION':
      if (selectedCollections.length === 0) {
        Alert.alert(t('common.hint'), t('share.create.pleaseSelectCollection'));
        return;
      }
      defaultTitle = t('share.create.collectionShare', { count: selectedCollections.length });
      data.collectionIds = selectedCollections;
      break;
    case 'LIST':
      if (selectedLists.length === 0) {
        Alert.alert(t('common.hint'), t('share.create.pleaseSelectGroup'));
        return;
      }
      defaultTitle = selectedLists.length === 1
        ? lists?.find((l) => l.id === selectedLists[0])?.name || t('share.create.groupShare')
        : t('share.create.groupsShare', { count: selectedLists.length });
      data.listIds = selectedLists;
      break;
    case 'TAG':
      if (selectedTags.length === 0) {
        Alert.alert(t('common.hint'), t('share.create.pleaseSelectTag'));
        return;
      }
      defaultTitle = selectedTags.length === 1
        ? `#${tags?.find((t) => t.id === selectedTags[0])?.name}` || t('share.create.tagShare')
        : t('share.create.tagsShare', { count: selectedTags.length });
      data.tagIds = selectedTags;
      break;
  }
}

import React, { useEffect, useState, useRef, useCallback } from 'react';
```

```
import {
  View,
  Text,
  FlatList,
  TouchableOpacity,
  Linking,
  Share,
  Alert,
  TextInput,
  Modal,
  ActivityIndicator,
} from 'react-native';
import { useRoute, useNavigation, RouteProp, useFocusEffect } from '@react-navigation/native';
import { Ionicons } from '@expo/vector-icons';
import { useMutation, useQueryClient } from '../lib/react-query';
import * as Clipboard from 'expo-clipboard';
import AsyncStorage from '@react-native-async-storage/async-storage';
import Toast from 'react-native-toast-message';
import LazyImage from '../components/LazyImage';
import { useThemeStore } from '../store/theme';
import { api, getPublicBaseUrl, recordShareView } from '../lib/api';
import { getPlatformName, getPlatformColor, getPlatformIcon, getPlatformConfig, buildDeepLink }
from '../lib/platforms';
import { useI18n } from '../lib/i18n';
import { ShareDetailSkeleton } from '../components/SkeletonComponents';
import StarRating from '../components/StarRating';

interface Collection {
  id: string;
  title: string;
  coverImage: string | null;
  platform: string;
  url?: string;
  rating?: number | null;
}

interface ShareData {
  id: string;
  title: string;
  description: string | null;
  hasPassword: boolean;
  needsPassword?: boolean;
  password?: string;
  shareUrl?: string;
```

```
isOwner: boolean;
isExpired: boolean;
expiresAt: string | null;
viewCount?: number;
collections: Collection[];
}

type ShareDetailRouteProp = RouteProp<{ ShareDetail: { shareId: string; isOwner?: boolean;
password?: string } }, 'ShareDetail'>;

export default function ShareDetailScreen() {
  const route = useRoute<ShareDetailRouteProp>();
  const navigation = useNavigation();
  const queryClient = useQueryClient();
  const { colors } = useThemeStore();
  const { t } = useI18n();
  const { shareId, isOwner: navIsOwner, password: navPassword } = route.params;

  const [data, setData] = useState<ShareData | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  const [isExpired, setIsExpired] = useState(false);
  const [isOwner, setIsOwner] = useState(navIsOwner || false);

  const [isPasswordVerified, setIsPasswordVerified] = useState(false);
  const [verifiedPassword, setVerifiedPassword] = useState('');
  const [passwordInput, setPasswordInput] = useState('');
  const [passwordError, setPasswordError] = useState('');
  const [verifying, setVerifying] = useState(false);

  const [savePasswordModal, setSavePasswordModal] = useState(false);
  const [savePasswordInput, setSavePasswordInput] = useState('');
  const [savePasswordError, setSavePasswordError] = useState('');

  const [snackbar, setSnackbar] = useState<{ visible: boolean; message: string; undold?:
string }>({ visible: false, message: '' });
  const undoTimerRef = useRef<NodeJS.Timeout | null>(null);

  const isLocked = data?.hasPassword && !isOwner && !isPasswordVerified;

  const fetchShareData = useCallback(async (isMounted = true) => {
    setLoading(true);
    setError('');
    try {
```

```
const response = await api.get(`/s/${shareId}`);
if (!isMounted) return;
const result = response.data;

if (result.isExpired) {
  setIsExpired(true);
  setData(result);
  setLoading(false);
  return;
}

if (result.isOwner) {
  setData(result);
  setIsOwner(true);
  setLoading(false);
  return;
}

if (navIsOwner) {
  setIsOwner(true);
  setData({
    ...result,
    isOwner: true,
  });
  setLoading(false);
  return;
}

setData(result);
setIsOwner(result.isOwner || false);
recordShareView(shareId).catch(() => {});
if (result.hasPassword && !result.isOwner) {
  try {
    const savedPassword = await
AsyncStorage.getItem(`linkchest-share-pwd-${shareId}`);
    if (savedPassword) {
      const verifyRes = await api.post(`/s/${shareId}/verify`, { password:
savedPassword });
      setData((prev: any) => ({ ...prev, ...verifyRes.data, needsPassword: false }));
      setIsPasswordVerified(true);
      setVerifiedPassword(savedPassword);
    }
  } catch {
  }
}
```

```
    }
  } catch (err: any) {
    if (!isMounted) return;
    setError(err.response?.data?.message || err.response?.data?.error || err.message ||
t('share.view.loadFailed'));
  } finally {
    if (isMounted) setLoading(false);
  }
}, [shareId, navIsOwner, t]);

useFocusEffect(useCallback(() => {
  let isMounted = true;
  fetchShareData(isMounted);
  return () => { isMounted = false; };
}, [fetchShareData]));

const handleVerifyPassword = async () => {
  if (!passwordInput.trim()) {
    setPasswordError(t('share.view.pleaseEnterPassword'));
    return;
  }
  setVerifying(true);
  setPasswordError("");
  try {
    const response = await api.post(`/s/${shareId}/verify`, { password:
passwordInput.trim() });
    const verifiedData = response.data;
    setData({
      ...data!,
      ...verifiedData,
      needsPassword: false,
    });
  }
}

import React, { useState, useCallback } from 'react';
import {
  View,
  Text,
  FlatList,
  TouchableOpacity,
  Alert,
  Share,
  RefreshControl,
  Modal,
  TextInput,
```

```
    ActivityIndicator,
  } from 'react-native';
import { useNavigation, NavigationProp, useFocusEffect } from '@react-navigation/native';
import { Ionicons } from '@expo/vector-icons';
import { useQuery, useMutation, useQueryClient } from '../lib/react-query';
import Toast from 'react-native-toast-message';
import * as Clipboard from 'expo-clipboard';
import { api, getPublicBaseUrl } from '../lib/api';
import { useThemeStore } from '../store/theme';
import { useI18n } from '../lib/i18n';
import { logEvent } from '../lib/analytics';
import { ShareManagementSkeleton } from '../components/SkeletonComponents';

type RootStackParamList = {
  Main: undefined;
  ShareDetail: { shareId: string; isOwner?: boolean; password?: string };
  CreateShare: undefined;
  CollectionDetail: { id: string };
  Tags: undefined;
  Lists: undefined;
};

interface ShareItem {
  id: string;
  title: string;
  type: 'ALL' | 'LIST' | 'TAG' | 'MULTI_TAG' | 'MULTI_LIST' | 'CUSTOM' | 'COLLECTION';
  isActive: boolean;
  createdAt: string;
  itemCount: number;
  viewCount?: number;
  hasPassword?: boolean;
  password?: string;
  shareUrl?: string;
  description?: string;
}

interface ShareList {
  id: string;
  name: string;
  description: string | null;
  collectionCount: number;
  createdAt: string;
}
```

```
const SHARE_DESC_PREFIXES = ['来自分享:', 'From share:'];
const isFromShare = (item: any) => {
  if (item.sourceShareId && item.sourceType === 'import') return true;
  const desc = item.description;
  return desc ? SHARE_DESC_PREFIXES.some(p => desc.startsWith(p)) : false;
};

const extractShareId = (item: any) => {
  if (item.sourceShareId && item.sourceType === 'import') return item.sourceShareId;
  const desc = item.description;
  if (!desc) return "";
  for (const p of SHARE_DESC_PREFIXES) {
    if (desc.startsWith(p)) return desc.slice(p.length).trim();
  }
  return "";
};

export default function ShareManagementScreen() {
  const navigation = useNavigation<NavigationProp<RootStackParamList>>();
  const queryClient = useQueryClient();
  const { colors } = useThemeStore();
  const { t } = useI18n();

  const typeLabels: Record<string, string> = {
    ALL: t('share.typeAll'),
    LIST: t('share.typeGroup'),
    TAG: t('share.typeTag'),
    MULTI_TAG: t('share.typeMultiTag'),
    MULTI_LIST: t('share.typeMultiGroup'),
    CUSTOM: t('share.typeCustom'),
    COLLECTION: t('share.typeCollection'),
    COLLECTIONS: t('share.typeCollections'),
  };

  const [activeTab, setActiveTab] = useState<'created' | 'received'>('created');
  const [refreshing, setRefreshing] = useState(false);
  const [linkInputModal, setLinkInputModal] = useState(false);
  const [linkInput, setLinkInput] = useState("");
  const [linkLoading, setLinkLoading] = useState(false);
  const [revealedPasswords, setRevealedPasswords] = useState<Record<string, string>>({});

  const { data: shares, isLoading: isLoadingShares, refetch: refetchShares } = useQuery({
    queryKey: ['shares'],
    queryFn: async () => {
      const response = await api.get('/shares');
      return (response.data.data || response.data) as ShareItem[];
    }
  });
```

```
    },
  });

  const { data: shareLists, isLoading: isLoadingShareLists, refetch: refetchShareLists } =
  useQuery({
    queryKey: ['share-lists'],
    queryFn: async () => {
      try {
        const response = await api.get('/lists');
        const allLists = (response.data.data || response.data) as any[];
        return allLists.filter((l: any) => isFromShare(l)) as ShareList[];
      } catch {
        return [] as ShareList[];
      }
    },
  });

  const onRefresh = async () => {
    setRefreshing(true);
    await Promise.all([refetchShares(), refetchShareLists()]);
    setRefreshing(false);
  };

  useFocusEffect(useCallback(() => {
    refetchShares();
    refetchShareLists();
  }, [refetchShares, refetchShareLists]));

  const deleteMutation = useMutation({
    mutationFn: (id: string) => api.delete(`/shares/${id}`),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['shares'] });
      Toast.show({ type: 'success', text1: t('share.management.shareLinkDeleted') });
    },
  });

  const toggleMutation = useMutation({
    mutationFn: (id: string) => api.put(`/shares/${id}/toggle`),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['shares'] });
    },
  });

import React, { useState } from 'react';
```

```
import {
  View,
  Text,
  TouchableOpacity,
  ScrollView,
  ActivityIndicator,
  Alert,
  Share as RNShare,
} from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import * as FileSystem from 'expo-file-system';
import { useThemeStore } from '../store/theme';
import { useI18n } from '../lib/i18n';
import { api } from '../lib/api';

type ExportFormat = 'json' | 'csv' | 'html';

interface FormatOption {
  key: ExportFormat;
  icon: string;
  ext: string;
  mimeType: string;
}

const FORMAT_OPTIONS: FormatOption[] = [
  { key: 'json', icon: 'code-slash-outline', ext: 'json', mimeType: 'application/json' },
  { key: 'csv', icon: 'grid-outline', ext: 'csv', mimeType: 'text/csv' },
  { key: 'html', icon: 'globe-outline', ext: 'html', mimeType: 'text/html' },
];

export default function ExportScreen() {
  const { colors } = useThemeStore();
  const { t } = useI18n();
  const [exporting, setExporting] = useState<ExportFormat | null>(null);

  const handleExport = async (format: ExportFormat) => {
    const opt = FORMAT_OPTIONS.find(o => o.key === format)!;
    setExporting(format);
    try {
      const response = await api.get(`/collections/export?format=${format}`, {
        responseType: format === 'json' ? 'json' : 'text',
      });
    }

    let content: string;
```

```
    if (format === 'json') {
      const exportData = response.data?.data || response.data;
      content = JSON.stringify(exportData, null, 2);
    } else {
      content = typeof response.data === 'string' ? response.data : String(response.data);
    }

    const filename = `linkchest-export.${opt.ext}`;
    const filePath = `${FileSystem.cacheDirectory}${filename}`;

    await FileSystem.writeAsStringAsync(filePath, content, {
      encoding: FileSystem.EncodingType.UTF8,
    });

    const fileInfo = await FileSystem.getInfoAsync(filePath);
    if (fileInfo.exists) {
      await RNShare.share({
        message: format === 'csv' ? content : content.substring(0, 60000),
        title: t('export.shareTitle'),
      });

      Alert.alert(
        t('common.success'),
        t('export.successMsg') + '\n\n' + t('export.filePath') + filePath
      );
    }
  } catch (error: any) {
    const msg = error?.response?.data?.message || error?.response?.data?.error ||
error?.message || t('common.operationFailed');
    Alert.alert(t('common.error'), msg);
  } finally {
    setExporting(null);
  }
};

return (
  <ScrollView style={{ flex: 1, backgroundColor: colors.background }}>
    <View style={{ padding: 16, gap: 16 }}>
      <View style={{ backgroundColor: colors.card, borderRadius: 12, padding: 16 }}>
        <Text style={{ fontSize: 13, fontWeight: '600', color: colors.textTertiary, textTransform:
'uppercase', letterSpacing: 0.5 }}>
          {t('export.formatGroupA')}
        </Text>
        <Text style={{ fontSize: 12, color: colors.textTertiary, marginTop: 4 }}>
```

```
      {t('export.formatGroupADesc')}}
    </Text>
    <View style={{ gap: 8, marginTop: 12 }}>
      <FormatButton
        icon={FORMAT_OPTIONS[0].icon}
        title="JSON"
        desc={t('export.jsonDesc')}
        loading={exporting === 'json'}
        colors={colors}
        onPress={() => handleExport('json')}
      />

      <FormatButton
        icon={FORMAT_OPTIONS[2].icon}
        title="HTML"
        desc={t('export.htmlDesc')}
        loading={exporting === 'html'}
        colors={colors}
        onPress={() => handleExport('html')}
      />
    </View>
  </View>

  <View style={{ backgroundColor: colors.card, borderRadius: 12, padding: 16 }}>
    <Text style={{ fontSize: 13, fontWeight: '600', color: colors.textTertiary, textTransform:
'uppercase', letterSpacing: 0.5 }}>
      {t('export.formatGroupB')}
    </Text>
    <Text style={{ fontSize: 12, color: colors.textTertiary, marginTop: 4 }}>
      {t('export.formatGroupBDesc')}
    </Text>
    <View style={{ gap: 8, marginTop: 12 }}>
      <FormatButton
        icon={FORMAT_OPTIONS[1].icon}
        title="CSV"
        desc={t('export.csvDesc')}
        loading={exporting === 'csv'}
        colors={colors}
        onPress={() => handleExport('csv')}
      />
    </View>
  </View>

  <View style={{ backgroundColor: colors.primary + '10', borderColor: colors.primary +
```

```
'30', borderWidth: 1, borderRadius: 12, padding: 14, flexDirection: 'row', alignItems: 'flex-start', gap: 10 }>
```

```
    <Ionicons name="information-circle-outline" size={20} color={colors.primary} />
    <View style={{ flex: 1 }}>
      <Text style={{ fontSize: 13, color: colors.text, fontWeight:
'500' }}>{t('export.hintTitle')}

```

```
function FormatButton({ icon, title, desc, loading, colors, onPress }: {
  icon: string;
  title: string;
  desc: string;
  loading: boolean;
  colors: any;
  onPress: () => void;
}) {
```

```
import React, { useState } from 'react';
import {
  View,
  Text,
  StyleSheet,
  TouchableOpacity,
  ScrollView,
  ActivityIndicator,
  Alert,
} from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import { useMutation, useQueryClient } from '../lib/react-query';
import { useAuthStore } from '../store/auth';
import { useThemeStore } from '../store/theme';
import { api } from '../lib/api';
import { useI18n } from '../lib/i18n';
```

```
interface DuplicateItem {
  id: string;
  title: string;
```

```
    url: string;
    platform: string;
    coverImage: string | null;
    createdAt: string;
  }

interface DuplicateGroup {
  type: 'url' | 'title';
  items: DuplicateItem[];
  similarity: number;
}

export default function DuplicateDetectScreen() {
  const { user } = useAuthStore();
  const { colors } = useThemeStore();
  const { t } = useI18n();
  const queryClient = useQueryClient();

  const [duplicateResults, setDuplicateResults] = useState<DuplicateGroup[] | null>(null);
  const [expandedGroup, setExpandedGroup] = useState<number | null>(null);
  const [keepIds, setKeepIds] = useState<Record<number, string>>({});
  const [scanning, setScanning] = useState(false);

  const userTier = (user as any)?.userTier || 'medium';
  const canUseDuplicate = userTier === 'heavy' || userTier === 'super';

  const scanDuplicates = async () => {
    if (!canUseDuplicate) {
      Alert.alert(t('common.hint'), t('settings.upgradeHint'));
      return;
    }

    setScanning(true);
    try {
      const res = await api.post('/collections/scan-duplicates');
      const groups = (res.data.data || res.data) as DuplicateGroup[];
      setDuplicateResults(groups);
      setExpandedGroup(null);
      setKeepIds({});
      if (groups.length === 0) {
        Alert.alert(t('common.success'), t('settings.noDuplicates'));
      }
    } catch (err: any) {
      Alert.alert(t('common.error'), err?.response?.data?.message ||
```

```
t('common.unknownError'));
  } finally {
    setScanning(false);
  }
};

const mergeMutation = useMutation({
  mutationFn: ({ keepId, removeIds }: { keepId: string; removeIds: string[] }) =>
    api.post('/collections/merge-duplicates', { keepId, removeIds }),
  onSuccess: () => {
    Alert.alert(t('common.success'), t('settings.mergeSuccess'));
    queryClient.invalidateQueries({ queryKey: ['collections'] });
    queryClient.invalidateQueries({ queryKey: ['stats'] });
    scanDuplicates();
  },
  onError: (err: any) => {
    Alert.alert(t('common.error'), err?.response?.data?.message ||
t('common.operationFailed'));
  },
});

const handleKeepNewest = (groupId: number) => {
  const group = duplicateResults?.[groupId];
  if (!group) return;
  const newest = [...group.items].sort(
    (a, b) => new Date(b.createdAt).getTime() - new Date(a.createdAt).getTime()
  )[0];
  if (!newest) return;
  const removeIds = group.items.filter((i) => i.id !== newest.id).map((i) => i.id);
  mergeMutation.mutate({ keepId: newest.id, removeIds });
};

const handleMerge = (groupId: number) => {
  const group = duplicateResults?.[groupId];
  if (!group) return;
  const keepId = keepIds[groupId];
  if (!keepId) {
    Alert.alert(t('common.hint'), t('settings.keep'));
    return;
  }
}

import React from 'react';
import { View, Text, ScrollView } from 'react-native';
import { Ionicons } from '@expo/vector-icons';
```

```
import { useQuery } from '../lib/react-query';
import { api } from '../lib/api';
import { getPlatformIcon, PLATFORMS } from '../lib/platforms';
import { useThemeStore } from '../store/theme';
import { use18n } from '../lib/i18n';
import { StatsSkeleton } from '../components/SkeletonComponents';

interface PlatformStat {
  platform: string;
  name: string;
  color: string;
  count: number;
}

export default function PlatformStatsScreen() {
  const colors = useThemeStore(s => s.colors);
  const { t } = use18n();

  const { data: stats, isLoading } = useQuery({
    queryKey: ['platformStats'],
    queryFn: async () => {
      const response = await api.get('/stats/platforms');
      return response.data.data as PlatformStat[];
    },
  });

  const totalCount = stats?.reduce((sum, s) => sum + s.count, 0) || 0;
  const maxCount = stats?.length ? stats[0].count : 0;

  return (
    <ScrollView style={{ flex: 1, backgroundColor: colors.background }}>
      <Text style={{ fontSize: 16, fontWeight: '600', color: colors.text, marginLeft: 12,
marginBottom: 8 }}>{t('platform.distribution')}</Text>
      <View style={{ backgroundColor: colors.card, marginHorizontal: 12, marginBottom: 12,
borderRadius: 12, overflow: 'hidden' }}>
        {isLoading ? (
          <StatsSkeleton colors={colors} />
        ) : stats && stats.length > 0 ? (
          stats.map((stat, index) => {
            const percentage = totalCount > 0 ? (stat.count / totalCount * 100) : 0;
            const barWidth = maxCount > 0 ? (stat.count / maxCount * 100) : 0;
            return (
              <View key={stat.platform} style={{ paddingHorizontal: 16, paddingVertical: 14,
borderBottomWidth: index < stats.length - 1 ? 1 : 0, borderBottomColor: colors.borderLight }}>
```

```

        <View style={{ flexDirection: 'row', alignItems: 'center', justifyContent:
'space-between', marginBottom: 8 }}>
          <View style={{ flexDirection: 'row', alignItems: 'center', gap: 8, flexShrink:
0 }}>
            <Ionicons
              name={getPlatformIcon(stat.platform) as any}
              size={20}
              color={stat.color}
            />
            <Text style={{ fontSize: 15, color: colors.text, fontWeight: '500' }}
numberOfLines={1}>{stat.name}</Text>
          </View>
          <View style={{ flexDirection: 'row', alignItems: 'center', gap: 8 }}>
            <Text style={{ fontSize: 14, fontWeight: '600', color:
colors.text }}>{stat.count}</Text>
            <Text style={{ fontSize: 12, color:
colors.textTertiary }}>{percentage.toFixed(1)}%</Text>
          </View>
        </View>
        <View style={{ height: 8, backgroundColor: colors.secondaryBg, borderRadius:
4, overflow: 'hidden' }}>
          <View style={{ width: `${barWidth}%`, height: '100%', borderRadius: 4,
backgroundColor: stat.color }} />
        </View>
      </View>
    );
  })
): (
  <View style={{ alignItems: 'center', paddingVertical: 48 }}>
    <View style={{ width: 72, height: 72, borderRadius: 36, backgroundColor:
colors.primaryBg, justifyContent: 'center', alignItems: 'center', marginBottom: 16 }}>
      <Ionicons name="stats-chart-outline" size={32} color={colors.primary} />
    </View>
    <Text style={{ fontSize: 18, fontWeight: '600', color: colors.text, marginBottom:
6 }}>{t('platform.noData')}</Text>
    <Text style={{ fontSize: 14, color: colors.textTertiary }}>开始收集收藏后即可查看
平台分布</Text>
  </View>
)
</View>
</ScrollView>
);
}

```

```
import React, { useState, useEffect } from 'react';
import {
  View,
  Text,
  StyleSheet,
  TouchableOpacity,
  ScrollView,
  ActivityIndicator,
  Alert,
} from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import { useMutation, useQuery } from '../lib/react-query';
import { useAuthStore } from '../store/auth';
import { useThemeStore } from '../store/theme';
import { api } from '../lib/api';
import { useI18n } from '../lib/i18n';

export default function AutoBackupScreen() {
  const { user } = useAuthStore();
  const { colors } = useThemeStore();
  const { t } = useI18n();

  const [backupFreq, setBackupFreq] = useState<'off' | 'weekly' | 'monthly'>('off');

  const userTier = (user as any)?.userTier || 'medium';
  const canUseBackup = userTier === 'heavy' || userTier === 'super';

  const { data: userSettings, isLoading: settingsLoading } = useQuery({
    queryKey: ['user-settings-backup'],
    queryFn: async () => {
      const r = await api.get('/users/settings');
      return (r.data.data || r.data) as Record<string, unknown>;
    },
  });

  useEffect(() => {
    if (userSettings) {
      const freq = userSettings.backupFrequency as 'off' | 'weekly' | 'monthly' | undefined;
      if (freq) setBackupFreq(freq);
    }
  }, [userSettings]);

  const saveBackupMutation = useMutation({
```

```

mutationFn: (data: { backupFrequency: string; backupFormat: string }) =>
  api.put('/users/settings', data),
onSuccess: () => {
  Alert.alert(t('common.success'), t('settings.backupSaved'));
},
onError: (error: any) => {
  const msg = error?.response?.data?.message || error?.response?.data?.error ||
error?.message || t('common.operationFailed');
  Alert.alert(t('common.error'), msg);
},
});

const [backingUp, setBackingUp] = useState(false);

const handleImmediateBackup = async () => {
  setBackingUp(true);
  try {
    const response = await api.post('/users/backup');
    const result = response.data?.data;
    Alert.alert(
      t('common.success'),
      `备份成功! \n 共备份 ${result?.count || 0} 条收藏\n 时间: ${new
Date(result?.timestamp || Date.now()).toLocaleString()}`
    );
  } catch (error: any) {
    const msg = error?.response?.data?.message || error?.response?.data?.error ||
error?.message || t('common.operationFailed');
    if (msg.includes('云端存储暂不可用')) {
      Alert.alert(t('common.hint'), '云端存储暂不可用, 请稍后重试');
    } else {
      Alert.alert(t('common.error'), msg);
    }
  } finally {
    setBackingUp(false);
  }
};

if (!canUseBackup) {
  return (
    <View style={[styles.container, { backgroundColor: colors.background }]}>
      <View style={[styles.emptyCard, { backgroundColor: colors.card }]}>
        <Ionicons name="lock-closed-outline" size={48} color={colors.textTertiary} />
        <Text
          style={[styles.emptyTitle, {
            color:
colors.text }]}>{t('settings.proRequired')}</Text>

```

```

        <Text style={{styles.emptyDesc, { color:
colors.textTertiary }}}>{t('settings.upgradeHint')}

```

```

const options: { value: 'off' | 'weekly' | 'monthly'; label: string; icon: string }[] = [
  { value: 'off', label: t('settings.backupOff'), icon: 'close-circle-outline' },
  { value: 'weekly', label: t('settings.backupWeekly'), icon: 'calendar-outline' },
  { value: 'monthly', label: t('settings.backupMonthly'), icon: 'calendar-clear-outline' },
];

```

```

return (
  <ScrollView style={{styles.container, { backgroundColor: colors.background }}}>
    <View style={{ padding: 16, gap: 16 }}>
      <View style={{styles.card, { backgroundColor: colors.card }}}>
        <Text style={{styles.sectionTitle, { color:
colors.textTertiary }}}>{t('settings.backupFrequency')}

```

```

import React, { useState, useCallback } from 'react';
import {
  View,
  Text,
  StyleSheet,
  TouchableOpacity,
  FlatList,
  Alert,
  RefreshControl,
} from 'react-native';
import { useInfiniteQuery, useMutation, useQueryClient } from '../lib/react-query';
import { Ionicons } from '@expo/vector-icons';
import { useThemeStore } from '../store/theme';
import { useI18n } from '../lib/i18n';
import { api } from '../lib/api';
import { getPlatformConfig } from '../lib/platforms';
import LazyImage from '../components/LazyImage';

```

```

interface TrashItem {
  id: string;
  url: string;
  title: string;
  coverImage: string | null;
  platform: string;
}

```

```
    deletedAt: string;
  }

export default function TrashScreen() {
  const { colors } = useThemeStore();
  const { t } = useI18n();
  const queryClient = useQueryClient();
  const [selectedIds, setSelectedIds] = useState<Set<string>>(new Set());

  const { data, fetchNextPage, hasNextPage, isFetchingNextPage, isLoading, refetch } =
    useInfiniteQuery({
      queryKey: ['trash'],
      queryFn: async ({ pageParam = 1 }) => {
        const res = await api.get(`/collections/trash?page=${pageParam}&limit=40`);
        return res.data;
      },
      getNextPageParam: (lastPage: any) =>
        lastPage?.pagination?.page < lastPage?.pagination?.totalPages
          ? lastPage.pagination.page + 1
          : undefined,
      staleTime: 30 * 1000,
      initialPageParam: 1,
    });

  const items: TrashItem[] = data?.pages.flatMap((p: any) => p.data) ?? [];

  const restoreMutation = useMutation({
    mutationFn: async (ids: string[]) => api.post('/collections/trash/restore', { ids }),
    onSuccess: (_data: any, ids: string[]) => {
      Alert.alert(t('common.success'), t('collection.trash.restoreSuccess', { count: ids.length }));
      setSelectedIds(new Set());
      queryClient.invalidateQueries({ queryKey: ['trash'] });
      queryClient.invalidateQueries({ queryKey: ['collections'] });
    },
    onError: () => Alert.alert(t('common.error'), t('collection.trash.restoreSuccess', { count: 0 })),
  });
```